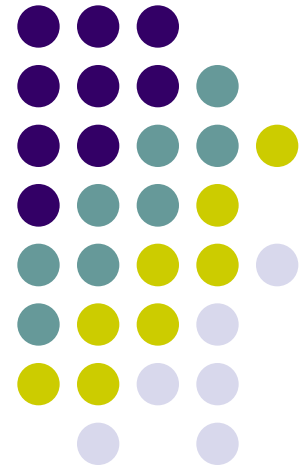
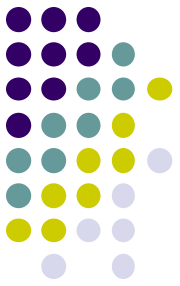


Natural Language Processing

CPSC 488

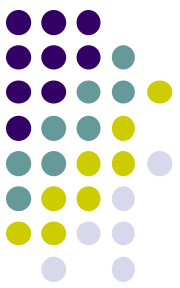
Probabilistic Language Modeling





Lecture Overview

- Language modeling
 - Concepts of language model
 - N-gram Language Models
 - Smoothing methods to generalize language models
 - Some practical issues in language modeling
- Evaluation of language models
 - Perplexity measure



Language Model (LM)

- **The language modeling problem**

- Given **finite vocabularies** and **infinite** set of **word sequences**,
- Assign a probability to a word/character, word sequence/character strings

$$P(W) = P(w_1, w_2, \dots, w_n)$$

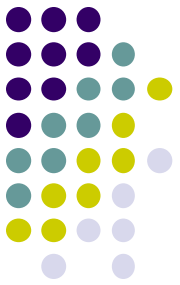
- **Language Model**

- A **probabilistic model** that **computes** the **probabilities** of words or word sequences
 - Modern language modeling is basically probabilistic language modeling.
- What does it mean by assigning a probability to a word or word sequence?
 - **P**("disseminating so much currency") = 10^{-15}
 - **P**("spending so much currency") = 10^{-9}

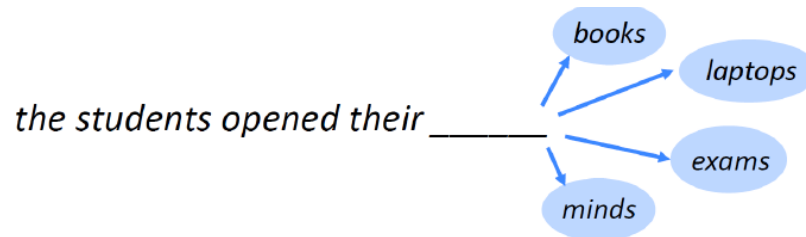
- **What can we do with the probabilistic model?**

- We can predict the next word.
- We can generate a sentence.
- and more NLP tasks

Predicting the Next Word with LM



- The problem of predicting the next word



- More formally, given a sequence of words $w_1, w_2, \dots, w_n$, **compute** the **probability distribution** of the **next word** w_{n+1} :

$$P(w_{n+1} | w_1, w_2, \dots, w_n) \text{ where } w_{n+1} \in \text{vocabulary } V = \{w_1, \dots, w_{|V|}\}$$

- Assume we computed the following probabilities:

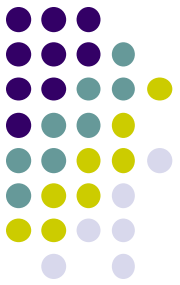
$$P(\text{books} \mid \text{the students opened their}) = 0.40$$

$$P(\text{laptops} \mid \text{the students opened their}) = 0.65$$

$$P(\text{exams} \mid \text{the students opened their}) = 0.10$$

$$P(\text{minds} \mid \text{the students opened their}) = 0.01$$

- Which word may come next given “the students opened their”?



What can we do with a LM?

- **Generate words or sentences**

- Generate words or sentences: $P(w_n | w_1, w_2, \dots, w_{n-1})$
 - “The students opened their [computers]”

- **Check spelling errors**

- By estimating the probability of a next word
 - E.g., For a sentence “The office is about fifteen minuets from my house.”
 - $P(\text{“about fifteen minutes from my house”}) > P(\text{“about fifteen minuets from my house”})$

- **Estimate the probability of correct words**

- Useful in speech recognition
- $P(\text{“I saw a van”}) \gg P(\text{“eyes awe of an”})$



What can we do with LM?

- **Rank possible sentences**

- By estimate the probability of a sentence in a natural language
- Sentence probabilities help decide word choice and order.
- Useful for grammar check, style check, and machine translation
- Examples
 - $P(\text{"strong winds"}) > P(\text{"large winds"})$
 - $P(\text{"Today is Wednesday"}) > P(\text{"Wednesday Today is"})$
 - $P(\text{"She is going home"}) > P(\text{"She is traveling to home"}) > P(\text{"To home she is going"})$

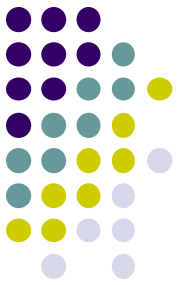
- **Language and grammar**

- Is a language created based on a grammar?
 - A sentence can be grammatical (and meaningful), but it may never occur in a corpus or never being used.
- What is grammar?
 - Is it a true probabilistic language model.

- **Language model has many other applications**

- Question-answering, handwriting recognition, etc.

Example Use of Language Models



Recommending next search keywords



what is the |

- what is the **weather**
- what is the **meaning of life**
- what is the **dark web**
- what is the **xfl**
- what is the **doomsday clock**
- what is the **weather today**
- what is the **keto diet**
- what is the **american dream**
- what is the **speed of light**
- what is the **bill of rights**

Google Search I'm Feeling Lucky

Smart reply

5:00

vacation plans

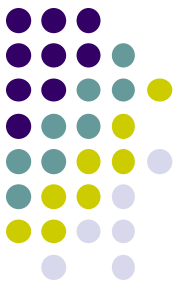
Peter Harbison
to me 5:00 PM

I think I'll be able to take a week of vacation next month. Leaning towards Kauai.

Do you have your vacation plans set yet? When you do, can you send them along?

Reply

No plans yet. I just sent them to you. I'm working on them.



Example Use of Language Models

- **Text generation** (<https://pdos.csail.mit.edu/archive/scigen/>)

Deploying Superblocks and Compilers

Julia and Dan

Abstract

Recent advances in replicated algorithms and relational symmetries have paved the way for architecture. After years of natural research into erasure coding, we show the deployment of courseware, which embodies the key principles of steganography. *Loy*, our new system for the exploration of sensor networks, is the solution to all of these issues.

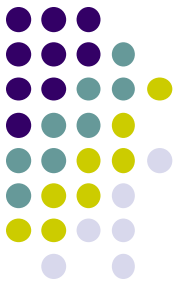
1 Introduction

Steganographers agree that robust symmetries are an interesting new topic in the field of cryptography, and information theorists concur. We view operat-

thesize unstable algorithms, we fulfil without investigating the evaluation of

Our contributions are threefold. First, how erasure coding can be applied to reinforcement learning. We present an algorithm for the deployment of extremizing (*Loy*), which we use to prove that and operating systems [19, 7, 14] can fill this goal. We examine how replication can be applied to the deployment of linked

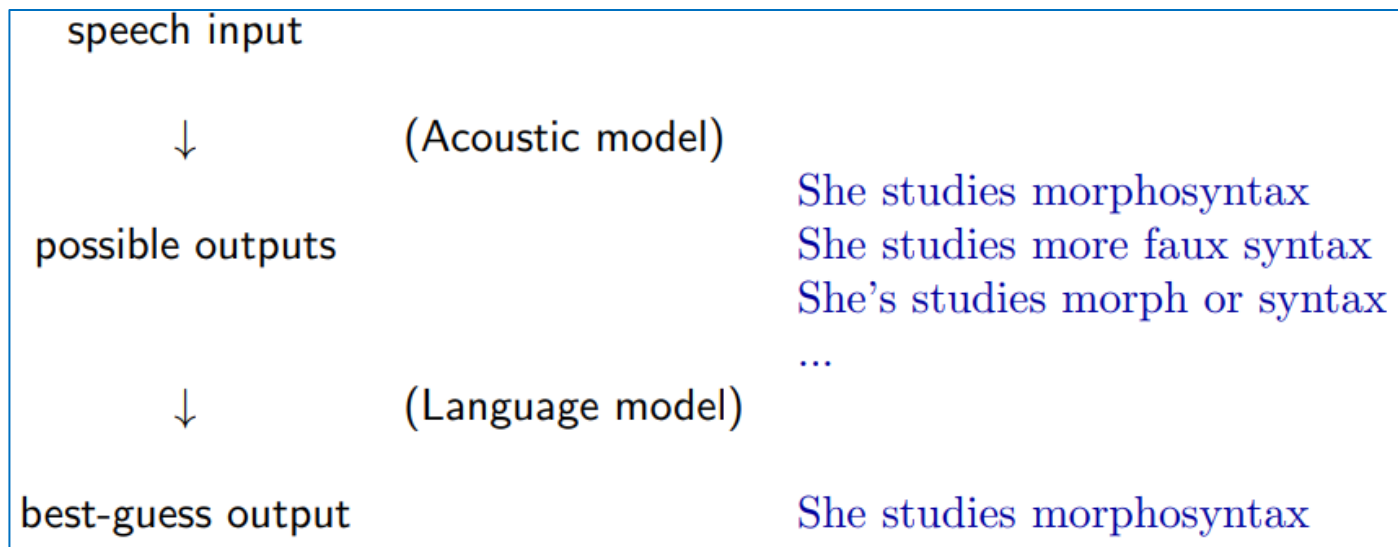
The rest of this paper is organized as follows. First, we motivate the need for fiber. We demonstrate the synthesis of the Turing machine. Finally, we conclude.



Example Use of Language Models

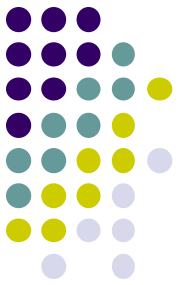
- **Speech recognition**

- Sentence probabilities help decide between similar-sounding words.



- **OCR/Handwriting recognition**

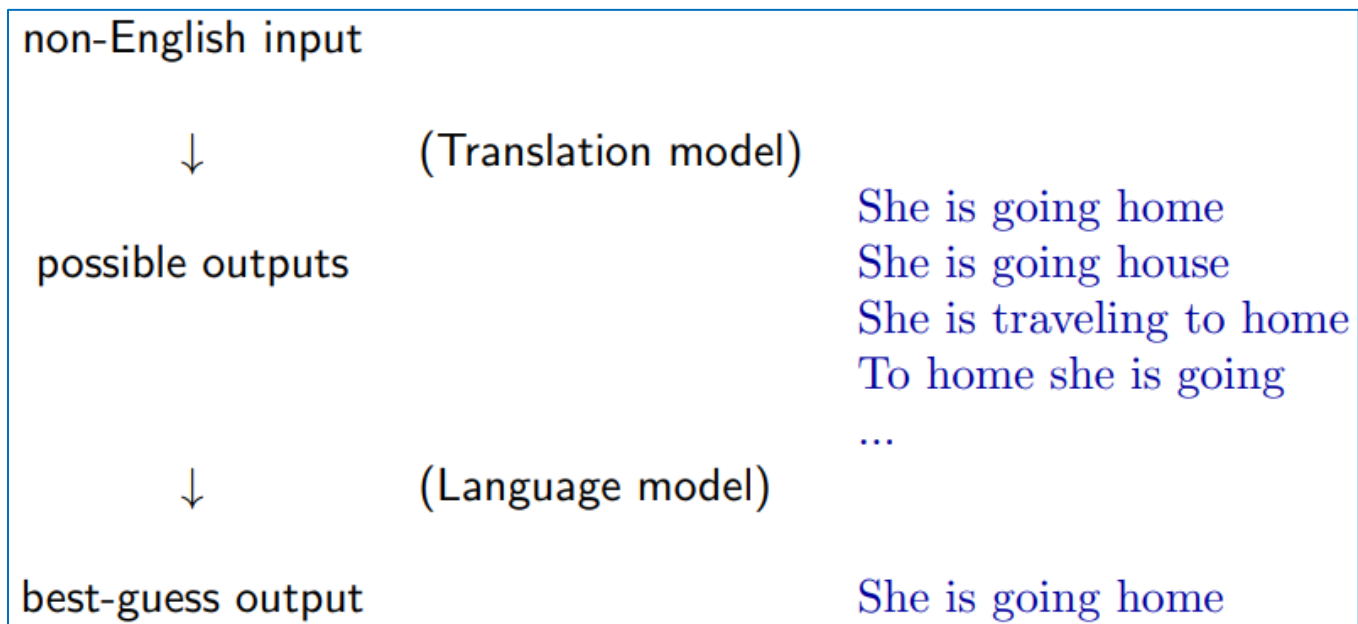
- Federal farm aid
- Federal form aid



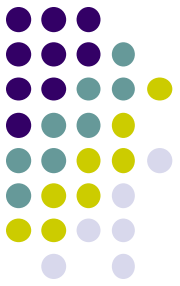
Example Use of Language Models

- **Machine translation**

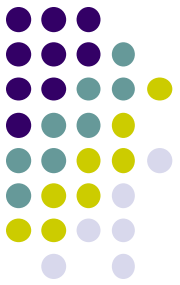
- Sentence probabilities help decide word choice and order.



Language Models Play the Role of...



- A judge of **grammatically**
- A judge of **semantic plausibility**
- An enforcer of **stylistic consistency**
- A repository of **knowledge** (?)



Language Modeling for NLP Tasks

- A **benchmark task** that helps us **measure** our **progress** on “**understanding a natural language**”
- **LM is a subcomponent of many NLP tasks:**
 - Predictive typing and recommendation of words for search
 - Speech recognition (from acoustic signal)
 - OCR/Handwriting recognition
 - Spelling/grammar correction
 - Authorship identification
 - Machine translation
 - Summarization
 - Text generation
 - Dialogue system
 - Etc.
- **Nowadays, Language Model is standard for NLP!**

Computing the Probability $P(W)$



- **How to compute the probability of a word sequence**

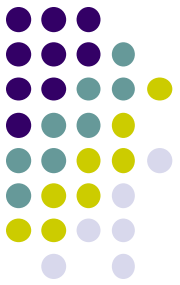
- $P(w_1, w_2, \dots, w_n) \Rightarrow$ joint probability (difficult to compute!)
- A trivial model
 - Assume we have N training sentences,
 - Let $w_1, w_2, \dots, w_n$ be a sentence, and $\text{count}(w_1, w_2, \dots, w_n)$ be the number of times it appeared in the training data.
 - Then, define a language model $P(w_1, w_2, \dots, w_n) = \frac{\text{count}(w_1, w_2, \dots, w_n)}{N}$
 - No generalization!

- **Better model**

- Given a sequence of n random variables, estimate a sequence probability by conditional probabilities.
- Conditional probability using the Chain Rule (changes joint to the conditional probabilities)

$$\begin{aligned} P(w_1, w_2, \dots, w_n) &= P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \dots P(w_n|w_1, w_2, \dots, w_{n-1}) \\ &= \prod_i^n P(w_i|w_1 w_2 \dots w_{i-1}) \end{aligned}$$

$$\begin{aligned} P(\text{its water is so transparent}) &= P(\text{its}) \times P(\text{water} \mid \text{its}) \times P(\text{is} \mid \text{its water}) \\ &\quad \times P(\text{so} \mid \text{its water is}) \times P(\text{transparent} \mid \text{its water is so}) \end{aligned}$$



Estimating Conditional Probabilities

- How to estimate the conditional probabilities
 - Count them: $P(w_{i+1}|w_i) = \frac{\text{count}(w_{i+1}, w_i)}{\text{count}(w_i)}$
 - $P(\text{the}|\text{its water is so transparent}) = \frac{\text{count}(\text{its water is so transparent the})}{\text{count}(\text{its water is so transparent})}$
 - But too many, $|V|^n$ possible sentences!
 - We'll never see enough data for estimating these.
 - Good news
 - The probability of a word only depends on a n previous words.
 - So, we can make an independent assumption (in fact, many joint probabilities are sparse).
- Markov assumption helps simplify the conditional probability estimation
 - $P(\text{the} | \text{its water is so transparent}) \approx P(\text{the} | \text{transparent})$ or
 - $P(\text{the} | \text{its water is so transparent}) \approx P(\text{the} | \text{so transparent})$

Markov Model

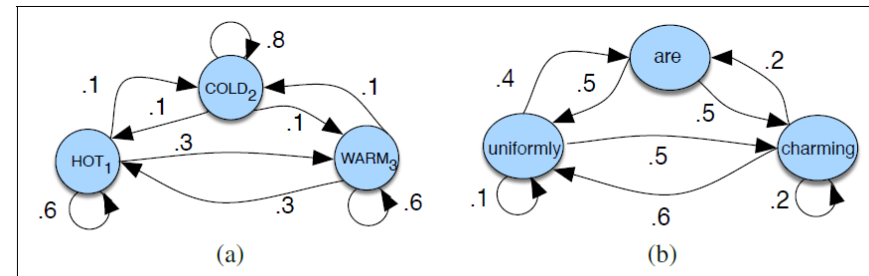


Figure A.1 A Markov chain for weather (a) and one for words (b), showing states and transitions. A start distribution π is required; setting $\pi = [0.1, 0.7, 0.2]$ for (a) would mean a probability 0.7 of starting in state 2 (cold), probability 0.1 of starting in state 1 (hot), etc.

- **Markov model** (Andrey Markov, 1906)
 - A **stochastic model** with the **Markov property**:
 - The **current/recent state(s)** is **sufficient statistics of the future**.
 - The **future** is **independent** of the past, **given the present/recent state(s)**.
 - This assumption is valid in so many real-world cases.



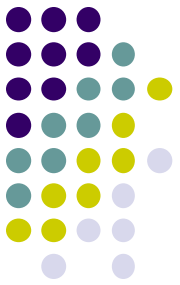
- **First-order Markov model**

- A state S_t is **Markov if and only if** $P(S_{t+1}|S_1, \dots, S_t) = P(S_{t+1}|S_t)$
 - The probability of next state S_{t+1} depends only on current state S_t .
- **Advantage** when **this assumption is valid**:
 - Calculation of joint probability becomes much simpler:

$$P(S_1, S_2, \dots, S_n) \approx \prod_i^n P(S_i|S_{i-1}) \approx P(S_{n-1}, S_n)$$

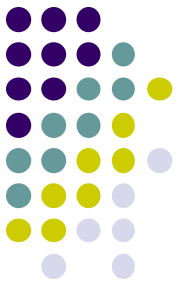
- **2nd order, ..., Kth-order Markov model**

- The **next state** only depends on the **k most recent states**.



Estimating $P(W)$ using Markov Model

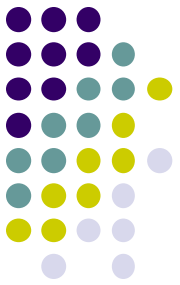
- Estimate $P(w)$ using **Zero-order Markov**
 - **unigram LM** – each word (the simplest case)
 - $P(w_1, w_2, \dots, w_n) \approx \prod_i^n P(w_i)$
- Estimate $P(w)$ using **1st-order Markov**
 - **bigram LM** – condition on the previous word
 - $P(w_i | w_1 w_2 \dots w_{i-1}) \approx \prod_i^n P(w_i | w_{i-1})$
- Estimate $P(w)$ using **2nd-order Markov**
 - **trigram LM** – condition on the most recent two previous words
 - $P(w_i | w_1 w_2 \dots w_{i-1}) \approx \prod_i^n P(w_i | w_{i-2}, w_{i-1})$
 -
- Estimate $P(w)$ using **Kth-order Markov**
 - **N-grams LM** – condition on the most recent K previous words
 - $P(w_1, w_2, \dots, w_n) \approx \prod_i^n P(w_i | w_{i-k}, \dots, w_{i-1})$ by the **Chain rule**
 - $P(w_i | w_1 w_2, \dots, w_{i-1}) \approx \prod_i^n P(w_i | w_{i-k}, \dots, w_{i-1})$



N-gram Language Models

- A **N-gram** is a chunk of **N consecutive words**.
- **Example:** “the students opened their...”
 - Unigram: “the”, “students”, “opened”, “their”
 - Bigram: “the students”, “students opened”, “opened their”
 - Trigram: “the students opened”, “students opened their”
 - 4-gram: “the students opened their”
 - ...
- **N-gram LM model** is a **(N-1)-order Markov model**.
 - Note: Markov model starts from zero while LM starts from one (unigram)

Google Books N-gram Viewer



Google Books Ngram Viewer

Q machine learning,natural language processing,artificial intelligence

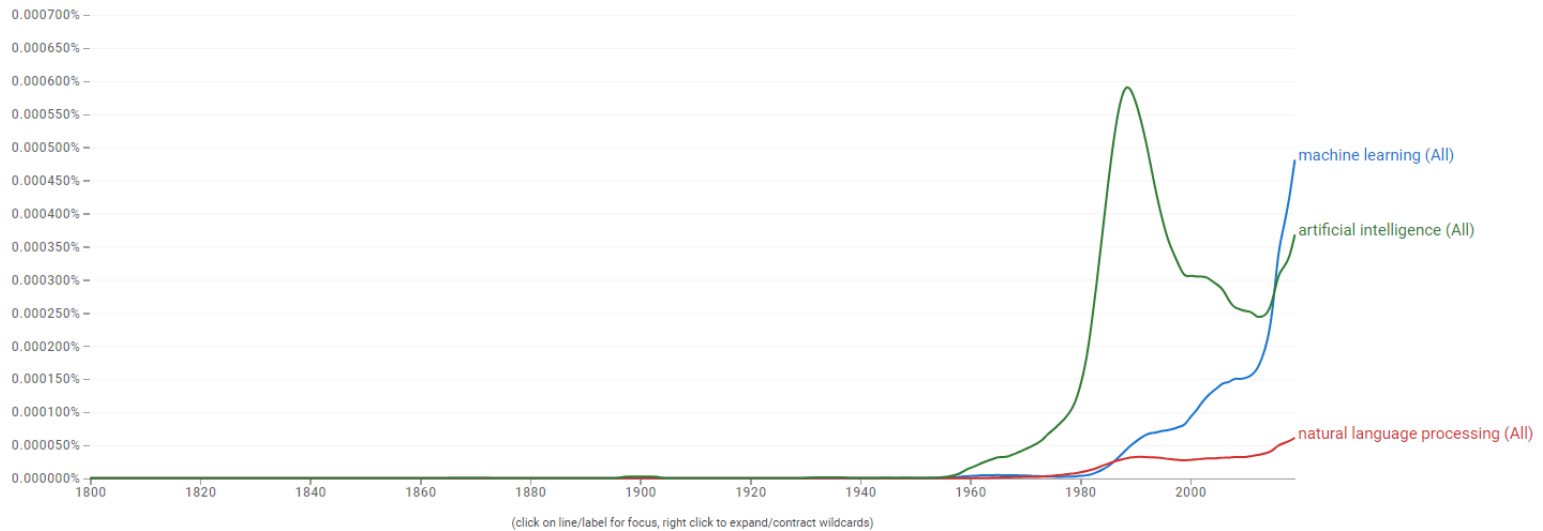


1800 - 2019

English (2019)

Case-Insensitive

Smoothing



Search in Google Books

machine learning

> 1800 - 1995

1996 - 2016

2017

2018

2019

English (2019)

natural language processing

>

1800 - 1985

1986 - 2011

2012 - 2015

2016 - 2018

2019

English (2019)

artificial intelligence

>

1800 - 1981

1982 - 1987

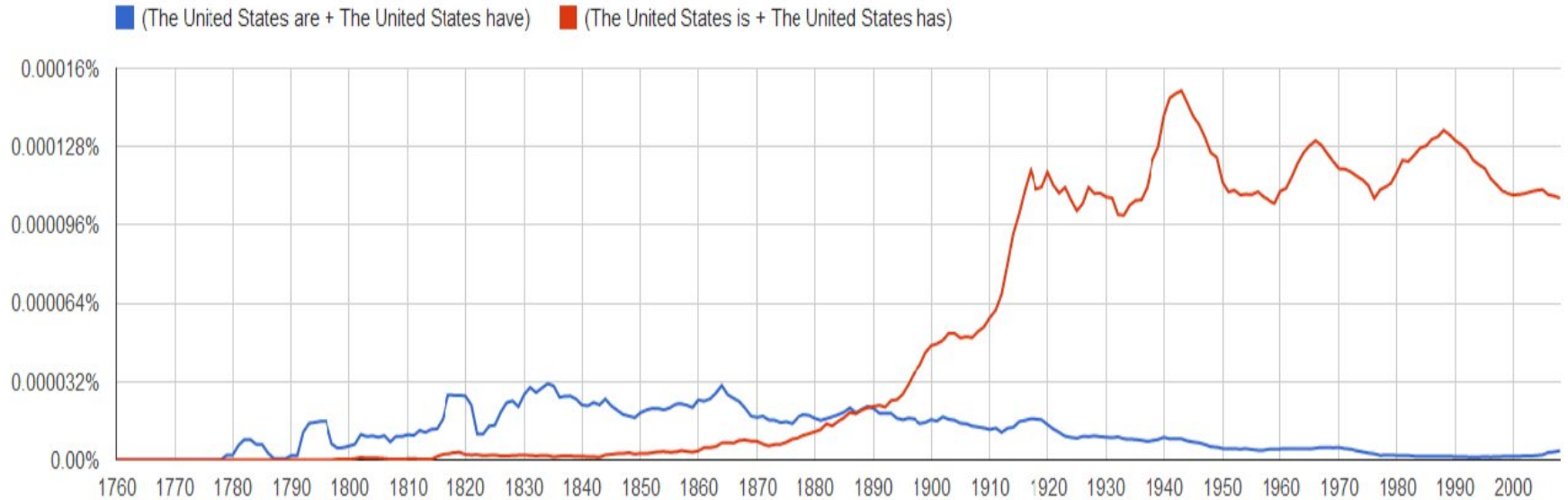
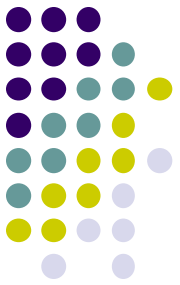
1988 - 1989

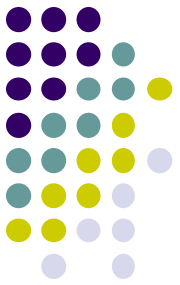
1990 - 2014

2015 - 2019

English (2019)

Usage of “United States” Over Time





Dealing with Variable Length Text

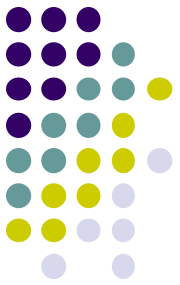
- **Assigning probabilities to variable length text**

- **Finite vocabulary** but **infinite set of sequences**
 - We want probability distribution over **sequences of any length** but...
- **Example:** Consider a trigram model for the following sentences

	w_1	w_2	w_3	w_4
(1)	he	saw	the	yellow
(2)	feeds	the	cats	daily

- **What's wrong in calculating probability for each word?**
 - Almost every probability is the same.
 - Does the trigram model capture these problems?
- **Use of special symbols** for a sentence “**beginning**” and “**end**”
 - Append a unique beginning and end symbol like **<s>** and **</s>** to every sentence and treat these symbols as **additional words**.
 - We now can assign probability distribution over all word sequences.

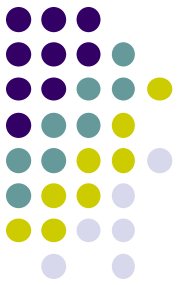
Beginning and End Symbols in a Sequence



	w_{-1}	w_0	w_1	w_2	w_3	w_4	w_5
(1)	<s>	<s>	he	saw	the	yellow	</s>
(2)	<s>	<s>	feeds	the	cats	daily	</s>

- Beginning <s> and end </s> symbols give the model a **context** of the **first** and **last word**.
 - E.g., “yellow” is an adjective followed by noun.
 - Therefore, the **probability** of “the yellow </s>” will be very **low** indicating this is **not a good sentence**.

Generating a N-gram Sentence



1. Prepare and preprocess a text data

- Collecting a corpus of text and preprocess it before building the N-gram model

2. Create N-grams

- Extracting all N-grams (N-words sequences) from the corpus

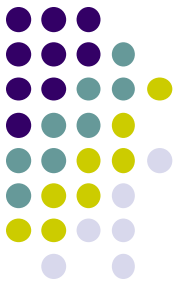
3. Calculate the probabilities of all N-grams

4. Generate the next word

- Beginning a sentence with a starting symbol **<s>** followed by (randomly or probabilistically) chosen first (N-1)-gram word(s).
- Based on the starting (N-1)-gram word(s), use the N-gram probabilities to predict the next word, e.g., for (“The”, “cat”), look for the most likely third word to follow.

5. Continue the process until stopping criterion

- Use the newly predicted word and the last word of the previous (N-1)-gram to form a new (N-1)-gram and predict the next word.
- Repeat this process to generate an entire sentence.
- Stopping criterion** can be text length or predicting the end token **</s>**.



Example: Trigram Probabilities

- **A trigram LM contains:**

- A vocabulary V
- A non-negative parameter (probability) $q(w|u, v)$ for **every** “trigram”, such that:

$$w \in V \cup \{< s >, </s >\} \text{ and } u, v \in V \cup \{< s >\}$$

- The probability of a sentence $(w_1, w_2, \dots, w_n)$ where $w_n = </s>$:

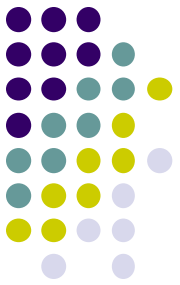
$$P(w_1, w_2, \dots, w_n) = \prod_i^n q(w_i | w_{i-1}, w_{i-2})$$

- **The probability of an example sentence** is calculated:

- $$\begin{aligned} P(<s> \text{ the dog barks } </s>) &= q(\text{the} \mid <s>, <s>) \times \\ &= q(\text{dog} \mid <s>, \text{the}) \times \\ &= q(\text{barks} \mid \text{the}, \text{dog}) \times \\ &= q(</s> \mid \text{dog}, \text{barks}) \end{aligned}$$

- **How can we choose the seed words w_{i-1}, w_{i-2} ?**

- Various methods possible



Estimating LM θ from Training Data

- How do we know the model $P(w|\theta)$?

- Use statistics from data (examples using Google N-grams)

- What if increasing N-gram order

- What is $P(\text{door} | \text{the})$?

$$\hat{P}(\text{door}|\text{the}) = \frac{14112454}{23135851162} = 0.0006.$$

- High order capture more dependencies: $\hat{P}(\text{door}|\text{close the}) = \frac{191125}{3785230} = 0.05.$

Bigram Model

198015222 the first
194623024 the same
168504105 the following
158562063 the world

...

14112454 the door

23135851162 the *

$$P(\text{door} | \text{the}) = \underline{0.0006}$$

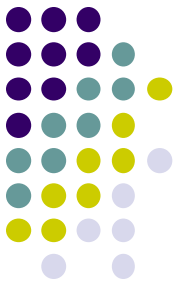
Trigram Model

197302 close the window 191125
close the door 152500 close the
gap 116451 close the thread
87298 close the deal

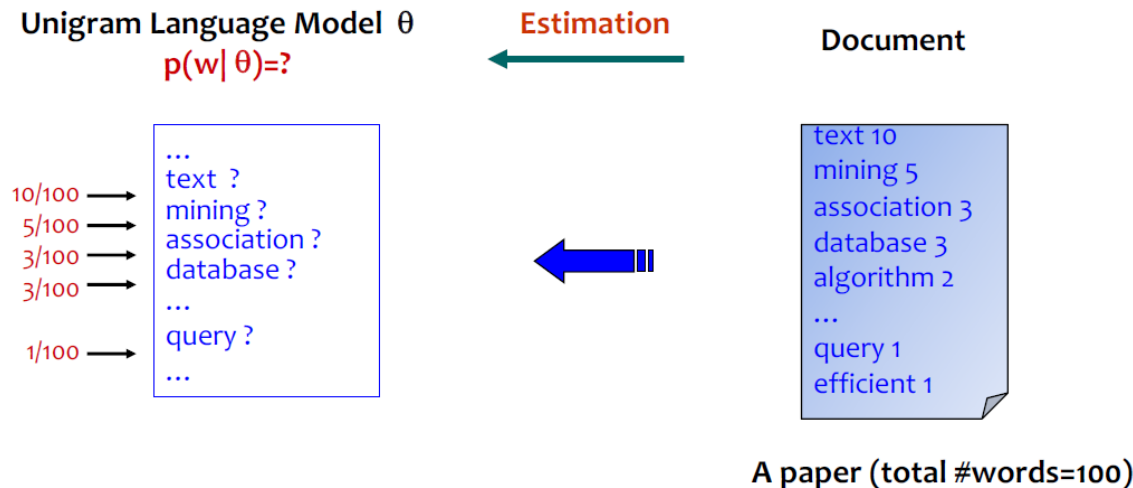
...

3785230 close the *

$$P(\text{door} | \text{close the}) = \underline{0.05}$$



Choosing Next Words

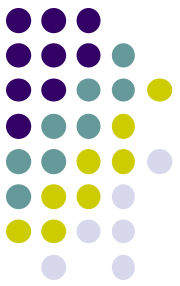


• How do we choose the next word?

- A simple idea: Collect statistics about how frequent different N-grams are and use them to predict the next word by MLE.

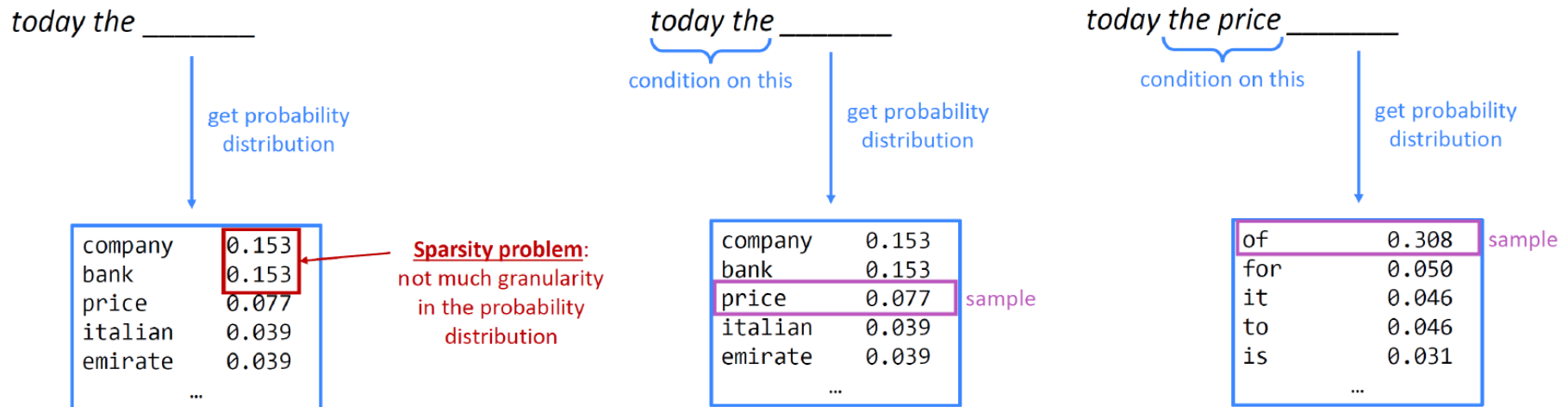
Maximum Likelihood Estimate $w_{MLE} = \operatorname{argmax}_w P(w_i|w_{i-1})$

- For practical reason: we do everything in the log space (to avoid underflow, faster compute)
- What may be the issue of MLE-based generation?

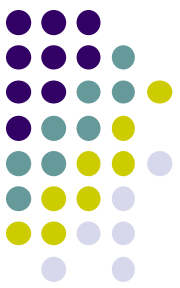


N-gram LMs in Practice

- You can **build a simple trigram LM** over a 1.7 million word corpus (e.g., Reuters) pretty quickly on your laptop.



- Some methods for selecting next word**
 - MLE and smoothing technique (to deal with zero-probability words)
 - Other more sophisticated methods
 - Sampling based methods, Beam search, etc. (will discuss more later)



Example: Berkeley Restaurant Project

- Some normalized sample user queries:

can you tell me about any good cantonese restaurants close by
mid priced thai food is what i'm looking for
tell me about chez panisse
can you give me a listing of the kinds of food that are available
i'm looking for a good place to eat breakfast
when is caffe venezia open during the day

- Raw bigram counts (The **sample words** chosen to cohere with each other)

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

The **most values** are **zero**.

A random set of eight words would be even **more sparse**.

Figure 3.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Zero counts are in gray.

Bigram Probabilities from Berkeley Restaurant Project



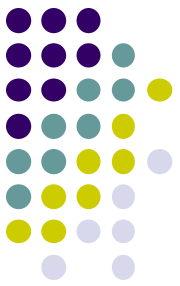
- **Word counts**

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- **Bigram probabilities** after normalization by the word counts

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Figure 3.2 Bigram probabilities for eight words in the Berkeley Restaurant Project corpus of 9332 sentences. Zero probabilities are in gray.



Using the Probability of a Sentence

- What is the probability of a sentence?

- **Bigram probabilities** for $P(<s>i \text{ want english food } </s>)$

$$= P(i | <s>)P(\text{want} | i)P(\text{english} | \text{want})P(\text{food} | \text{english})P(</s> | \text{food})$$

- **Calculated:** $P(i | <s>) = 0.25$, $P(\text{english} | \text{want}) = 0.0011$, $P(\text{food} | \text{english}) = 0.5$, $P(</s> | \text{food}) = 0.68$

- $P(<s>i \text{ want english food } </s>) = 0.25 \times 0.44 \times 0.0011 \times 0.5 \times 0.68 = \mathbf{0.000031}$

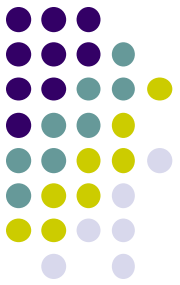
- **How about $P(<s>I \text{ want Chinese food } </s>)$?**

- What kinds of knowledge are captured? “**factual**” information?

- $P(\text{English} | \text{want}) = .0011$
- $P(\text{Chinese} | \text{want}) = .0065$
- $P(\text{to} | \text{want}) = .66$
- $P(\text{eat} | \text{to}) = .28$
- $P(\text{food} | \text{to}) = 0$
- $P(\text{want} | \text{spend}) = 0$
- $P(i | <s>) = .25$

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Figure 3.2 Bigram probabilities for eight words in the Berkeley Restaurant Project corpus of 9332 sentences. Zero probabilities are in gray.



Examples: Text Generation

- Some automatically generated sentences from a **unigram model**:

$$P(w_1, w_2, \dots, w_n) \approx \prod_i^n P(w_i)$$

fifth, an, of, futures, the, an, incorporated, a,
a, the, inflation, most, dollars, quarter, in, is,
mass

thrift, did, eighty, said, hard, 'm, july, bullish

that, or, limited, the

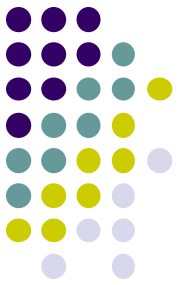
- Some text generated from a **bigram model**:

$$P(w_i | w_1 w_2, \dots, w_{i-1}) \approx \prod_i^n P(w_i | w_{i-1})$$

texaco, rose, one, in, this, issue, is, pursuing, growth, in,
a, boiler, house, said, mr., gurria, mexico, 's, motion,
control, proposal, without, permission, from, five, hundred,
fifty, five, yen

outside, new, car, parking, lot, of, the, agreement, reached

this, would, be, a, record, november



N-gram LMs in Practice

- **A text generated by Trigram**

today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .

Surprisingly grammatical!

...but **incoherent**. We need to consider more than three words at a time if we want to model language well.

But increasing n worsens sparsity problem,
and increases model size...

Approximating Shakespeare by N-gram Text Generation

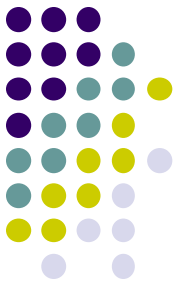


1 gram	–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have –Hill he late speaks; or! a more to leg less first you enter
2 gram	–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow. –What means, sir. I confess she? then all sorts, he is trim, captain.
3 gram	–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done. –This shall forbid it should be branded, if renown made it empty.
4 gram	–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in; –It cannot be but so.

- **Shakespeare as corpus**

- N=884,647 tokens, V=29,066
- What's coming out **looks like Shakespeare** because it is Shakespeare.

The Wall Street Journal is **not** like Shakespeare



1
gram

Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives

2
gram

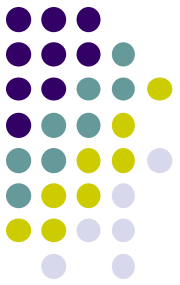
Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her

3
gram

They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

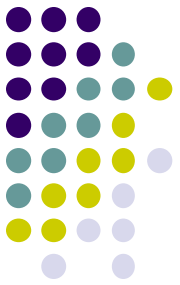
Why not?

What's the issue here?



Issue: Out-Of-Vocabulary (OOV)

- **Open vs closed vocabulary tasks**
 - Closed vocabulary task
 - If we know all the words in advanced, vocabulary V is fixed.
 - Open vocabulary task
 - Often, we don't know this => **Unknown** or **Out-Of-Vocabulary (OOV) words**
- **How to deal with OOV/unknown words:**
 - During training
 - Replace **all rare words** (appear below a threshold) with <UNK> or treat all words <UNK> except for the top-N most frequent words (**vocabulary cutoff**).
 - Compute N-gram probabilities including <UNK>
 - Now, every possible input sequence has some probability.
 - During testing/inference
 - Any unseen word is mapped to <UNK> and get the probabilities with <UNK> from training
 - Smoothing with <UNK> using various methods
 - Subword/character-level language modeling
 - **Less common** in N-gram LM but **common** in modern LM



Issue: Language Sparsity

- **The probabilities of unseen words or phrases**
 - By **MLE estimate**, the words not in the training data => **Zero probability**.
 - Training dataset is just a sample. Is it statistically a correct idea?
 - Example:
 - Shakespeare corpus consists of $N=884,647$ word tokens and vocabulary $V = 29,066$ word types.
 - Only about 30,000 word types occurred.
 - Only **0.04%** of **all possible bigrams** occurred.
- **The issues of sparsity in a language**
 - Most N-grams will never be observed, even if they are linguistically plausible.
 - Most sentences will have zero or undefined probabilities.

Approximating Shakespeare



1
gram

–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have

–Hill he late speaks; or! a more to leg less first you enter

2
gram

–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.

–What means, sir. I confess she? then all sorts, he is trim, captain.

3
gram

–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.

–This shall forbid it should be branded, if renown made it empty.

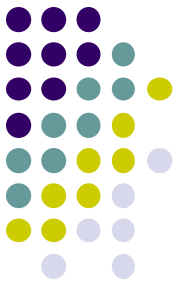
4
gram

–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;

–It cannot be but so.

● Shakespeare as corpus

- N=884,647 tokens, V=29,066
- Shakespeare produced 300,000 bigram types out of $V^2 = 844$ million possible bigrams.
- 99.96% of the possible bigrams were never seen (zero entries in the table).
- Quadrigrams worse – due to the sparsity of a language



Issue: The Perils of Overfitting

- **N-grams only work well** for word prediction if the **test corpus looks like the training corpus**.

- $P(\text{offer}|\text{denied the}) = 0$

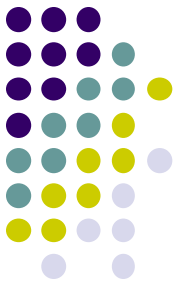
Training set:

... denied the allegations
... denied the reports
... denied the claims
... denied the request

Test set:

... denied the offer
... denied the loan

- **We need to train *robust models* that generalize!**
 - Things never occurred in the training data may occur in the test data.
 - One kind of generalization is **generalization of Zeros** (**smoothing**)



Choosing the Next Word by Context

- Suppose we are learning a 4-gram LM in a corpus.

- **Predict the next word for the following sentence:**

“As the proctor started the clock, the students opened their ____”

- **Computing the probabilities**

$$P(w | \text{students opened their}) = \frac{C(\text{students opened their } w)}{C(\text{students opened their})}$$

- **In the corpus,**

“students opened their” occurred 1000 times

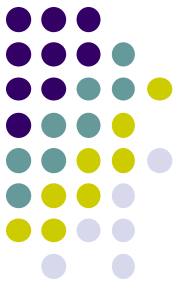
“students opened their books” occurred 400 times

- $\rightarrow P(\text{books} | \text{students opened their}) = 0.4$

“students opened their exams” occurred 100 times

- $\rightarrow P(\text{exams} | \text{students opened their}) = 0.1$

Should we have
discarded the
“proctor” context?



Issue: Long-Distance Dependencies

- **Problem**

- In many cases, **local context** does **not provide** the most useful predictive clues, which instead are provided by **long-distance dependencies**.
- “The computer which I had just put into the machine room on the fifth floor **crashed**.”

Syntactic dependencies

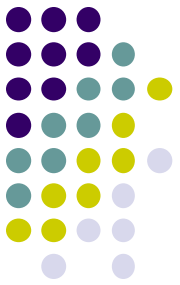
- “The *man* next to the large oak tree near the grocery store on the corner **is** tall.”
- “The *men* next to the large oak tree near the grocery store on the corner **are** tall.”

Semantic dependencies

- “The *bird* next to the large oak tree near the grocery store on the corner **flies** rapidly.”
- “The *man* next to the large oak tree near the grocery store on the corner **talks** rapidly.”

- **What we need:**

- More complex language models to handle such dependencies



Summary: N-gram LM Issues

- **The sparsity of language**

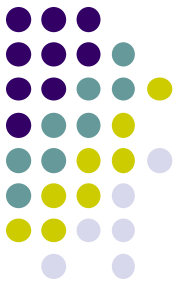
- The flaw of MLE
 - MLE makes the training data maximally probable, by making everything else (unseen data) minimally probable.
 - MLE assumes anything **not occurred** will **never occur**.
- Word probabilities are typically very small due to the sparsity of words.
- Many words may have zero or close to zero probabilities.
 - **Multiplying lots of small numbers** quickly causes **underflow**.
 - So, everything in **negative log space**.

- **Overfitting**

- N-grams only work well for word prediction if the test corpus looks like the training corpus.
 - In real life, it often doesn't. We need to train robust models that **generalize**!

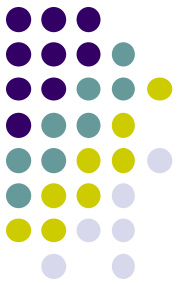
- **Not good enough to handle long-distance dependencies**

- Increasing the order may make the sparsity worse
- More sophisticated methods => **embeddings and large language models**

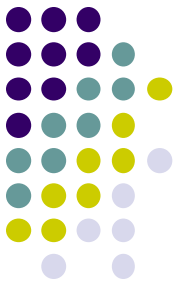


Toolkits for N-gram Language Models

- KenLM
- SRILM
- CMU Language Modeling Toolkit
- Gensim, Moses, NLTK, etc.
- Libraries for Kneser-Ney implementation



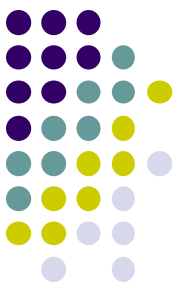
Smoothing Methods



Smoothing Methods

- Addressing the problems of sparsity and generalization
 - From a combinatorial number of possible word sequence, **many are rare** (but not impossible) **combinations never occur in training**,
 - But MLE incorrectly assign zero (a.k.a. **sparse data**, **infinite perplexity**).
 - **Basic idea**
 - By stealing probability mass from seen events and reallocating it to unseen events
 - Regularize or generalize!
- Many different *heuristic* smoothing methods
 - Add-one (Laplace)
 - Add- α
 - Good-Turing
 - Backoff and interpolation
 - Absolute discounting
 - Kneser-Ney
 - Class-based (cluster) N-grams
 - Etc.





The Intuition of Smoothing (from Dan Klein)

- When we have sparse statistics

$P(w \mid \text{denied the})$

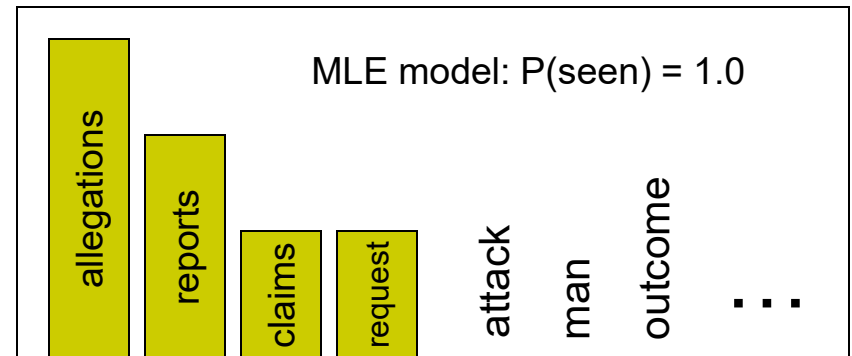
3 allegations

2 reports

1 claims

1 request

7 total



- Steal probability mass to generalize better

$P(w \mid \text{denied the})$

2.5 allegations

1.5 reports

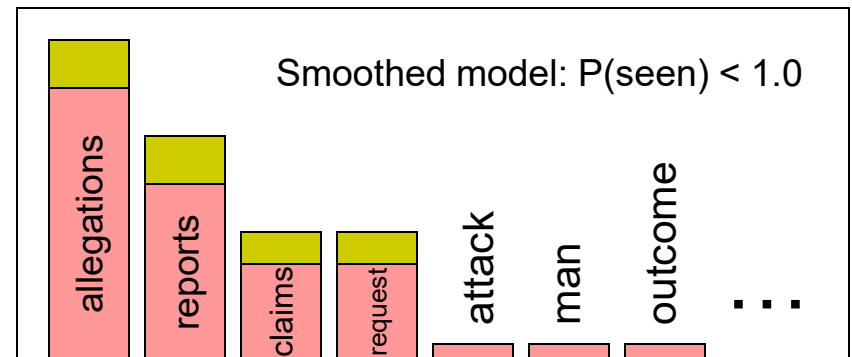
0.5 claims

0.5 request

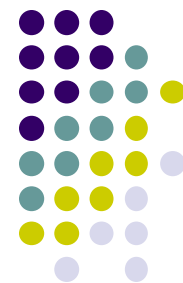
2 other

7 total

Adding probability mass to unseen events requires removing it from seen ones (discounting) in order to maintain the sum of a joint distribution = 1.



Smoothing Methods



- **Basic idea:**

- Pretend we saw each word one more time than we did and just add α to all the counts!

- From MLE estimate (**no smoothing**): $P(w_i|w_{i-1}) = \frac{C(w_i, w_{i-1})}{C(w_{i-1})}$

where $C(w_{i-1}) = \text{Count}(w_{i-1})$

- **Add-1 smoothing** (Laplace): $P(w_i|w_{i-1}) = \frac{C(w_i, w_{i-1}) + 1}{C(w_{i-1}) + |V|}$

- $|V|$ is vocabulary size (normalized by $|V|$).

- **Add- α smoothing** (Lidstone): $P(w_i|w_{i-1}) = \frac{C(w_i, w_{i-1}) + \alpha}{C(w_{i-1}) + \alpha \cdot |V|}$

- **Add-1** tends to reassign too much mass to unseen events, so can be adjusted to add $0 < \alpha < 1$ (normalized by $\alpha \cdot |V|$).

Add-1 Smoothed Bigrams from Berkeley Restaurant Corpus

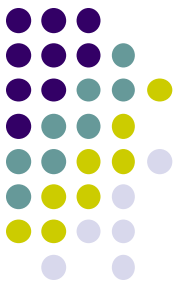


	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

$$P^*(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|} \text{ where } |V| = 1446$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058



Reconstituted Counts Matrix

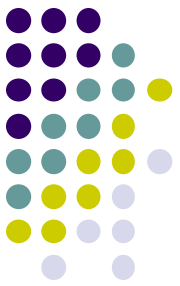
Add-1 smoothed bigram counts

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

$$C^*(w_i|w_{i-1}) = P^*(w_i|w_{i-1})C(w_{i-1}) = \frac{C(w_{i-1}w_i) + 1}{C(w_{i-1}) + |V|} C(w_{i-1})$$

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16



Comparing with Raw Bigram Counts

Raw Bigram counts

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Reconstituted counts matrix

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16



Issue: with Add-1 Smoothing

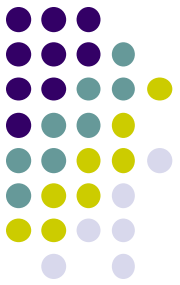
26 letter types	Actual frequencies		Add-1	
xya	1	1/3	2	2/29
xyb	0	0/3	1	1/29
xyd	0	0/3	1	1/29
xye	0	0/3	1	1/29
...				
xyz	0	0/3	1	1/29
Total xy	3	3/3	29	29/29

Suppose we have 20000-word types with more combinations of words.

The total count by Add-1 for “xyd” is 20003, then the probability is $3/20003$ and the probability of non-xyd events in training data $26/20003$ ($29 - 3$)=0.001.

But the number of novel events (events never occurred in training data) is 19998 ($20003 - 5$) and the total estimated probability of novel events is $19998/20003 = 0.999$.

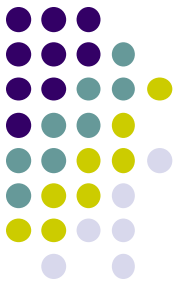
- Add-1 smoothing may work OK for text classification
 - Spam detection, sentiment analysis, but not good for language modeling
- Problems in language modeling:
 - Add-1 smoothing thinks we are extremely likely to see novel events rather than words we've actually seen => over-smoothing.
 - A large dictionary makes novel events too probable but reduces the probability of frequent events.



Add- α Smoothing

- **Add- α smoothing:** $P(w_i|w_{i-1}) = \frac{C(w_i, w_{i-1}) + \alpha}{C(w_{i-1}) + \alpha|V|}$
 - Add-1 ($\alpha=1$) tends to reassign too much mass to unseen events, so can be adjusted to add $0 < \alpha < 1$ (normalized by αV).

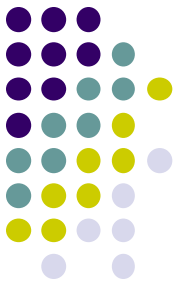
	Add-0.001 smoothing		add-1	add- α
xya	1	1/3	1.001	0.331
xyb	0	0/3	0.001	0.0003
xyc	0	0/3	0.001	0.0003
xyd	2	2/3	2.001	0.661
xye	0	0/3	0.001	0.0003
...				
xyz	0	0/3	0.001	0.0003
Total xy	3	3/3	3.026	1



How to Choose α

- How much probability to “**set aside**” for **novel events**?
 - It depends on how likely novel events really are.
 - Which may depend on the type of text, size of training corpus, ...
 - An idea of good-Turing smoothing
- Can we figure it out from the data?
 - Choose the α that **minimizes** **cross-entropy** through **training**.

Setting Smoothing Parameters from the Data



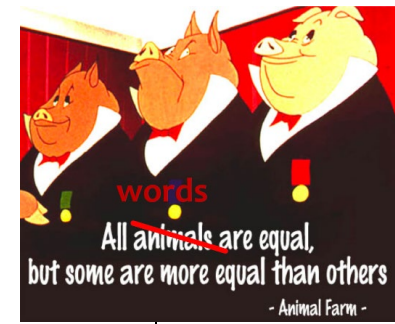
Training Data

Held-Out
Data

Test
Data

$$\log P(w_1 \dots w_n \mid M(\lambda_1 \dots \lambda_k)) = \sum_i \log P_{M(\lambda_1 \dots \lambda_k)}(w_i \mid w_{i-1})$$

- **Try many α values and report the one that gets best results**
 1. Three-way data split:
 - training set (80-90%), held-out set (5-10%), and test set (5-10%).
 2. Fix the N-gram probabilities (on the training data)
 3. Search for α that gives the largest probability from the held-out set.
 - Training the model to estimate probabilities with different values of α .
 4. Report the final results on the test set.
- **N-fold cross-validation (“Leave One Out”)**
 - Test each sentence with a smoothed model from other N-1 sentences.
 - We can reliably assess α .
 - Surprisingly fast: Usually easy to retrain by adding/subtracting counts.



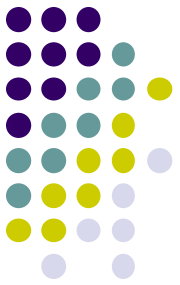
Backoff and Interpolation

- **Add-1 estimation is a blunt instrument.**
 - Not commonly used for N-grams but can be useful to smooth other NLP models such as text classification where the number of zeros isn't so huge.
 - Why are we treating all novel events as the same?
 - Backoff and Interpolation

- **Backoff**

- Sometimes, it helps to use less context. So, condition on less context for the contexts you haven't learned much about.
- Basic idea
 - If we have the trigram probability, $P(w_n|w_{n-1}, w_{n-2})$, then use it.
 - Otherwise “backoff” to the lower-order if no evidence for the higher order.
- Preference rule

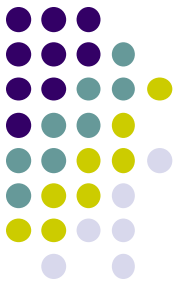
$$\hat{P}(w_i|w_{i-2}w_{i-1}) = \begin{aligned} &P(w_i|w_{i-2}w_{i-1}) \text{ if } C(w_{i-2}w_{i-1}w_i) \neq 0 \text{ else} \\ &\alpha_1 P(w_i|w_{i-1}) \text{ if } C(w_{i-1}w_i) \neq 0 \text{ else} \\ &\alpha_2 P(w_i) \end{aligned}$$



Backoff and Interpolation

$p(\text{zygote} \mid \text{see the})$ vs. $p(\text{baby} \mid \text{see the})$

- What if $\text{count}(\text{see the zygote}) = \text{count}(\text{see the baby}) = 0$?
- **baby** beats **zygote** as a unigram
- **the baby** beats **the zygote** as a bigram
- Does “**see the baby**” beat “**see the zygote**” as a trigram?
 - Even if both have the same count, **zero**.
- **Backoff and Interpolation**
 - Condition on less context for contexts you haven't learned much about
 - **Backoff**: use trigram if you have good evidence, otherwise bigram or unigram.
 - **Interpolation**: **mixture** of unigram, bigram, and trigram, etc. models
 - Interpolation generally works better.



Linear Interpolation

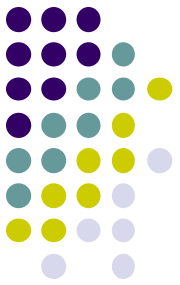
- **Simple interpolation through model combination**

- As **N** increases, the power of an **N-gram model** increases, but the ability to estimate accurate parameters from **sparse data** decreases.
- Combine the results of multiple N-gram models of increasing complexity.
- **Basic idea**: Linearly combine estimates of N-gram models of increasing order. Lambdas conditional on context.

$$\hat{P}(w_i|w_{i-2}w_{i-1}) = \lambda_1 P(w_i|w_{i-2}w_{i-1}) + \lambda_2 P(w_i|w_{i-1}) + \lambda_3 P(w_i)$$

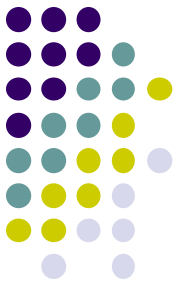
where $\sum \lambda_i = 1$

- **Learn proper values** for λ_i by **training** to approximately maximize the likelihood of an independent development (a.k.a tuning) corpus.
- **Interpolation works better than backoff.**



Example: Linear Interpolation

- Suppose we train unigram, bigram, and trigram LMs on the following corpus:
 - <s> I am Sam </s>
 - <s> Sam I am </s>
 - <s> I do not like green eggs and ham </s>
- What is $P(\text{Sam} \mid \text{I am})$ if we use linear interpolation with $\lambda_i = 1/3$?
 - $$P(\text{Sam} \mid \text{I am}) = \frac{1}{3}P(\text{Sam}) + \frac{1}{3}P(\text{Sam} \mid \text{am}) + \frac{1}{3}P(\text{Sam} \mid \text{I am})$$
$$= \frac{1}{3} \times \frac{2}{20} + \frac{1}{3} \times \frac{1}{2} + \frac{1}{3} \times \frac{1}{2}$$



More Smoothing



Good-Turing Smoothing



- **Basic idea**

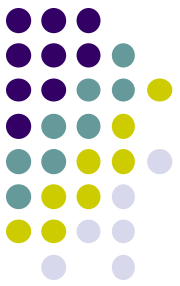
- Adjust the probability of **unseen events** based on the frequency of events seen only a few times.

- **Good-Turing estimate**

- For each N-gram, count frequency of it that occurred c times and adjust the count.
- Adjusted count $c^* = \frac{(c+1)N_{c+1}}{N_c}$ where N_c is the frequency N-gram that occurred c times. Each count is reduced slightly.
- Adjusted probability $P(x) = \frac{c^*}{N}$ by adjusted count where N is the total counts.
 - Example: $N_1 = 10$ (10 trigrams occurred once), $N_2 = 5$, $N_0 = 20$.
 - The adjusted count for n-gram occurred **once** $c^* = \frac{(1+1)5}{10} = 1$
 - The adjusted count **never occurred** $c^* = \frac{N_1}{N_0} = \frac{10}{20} = 0.5$
- It tries to **handle unseen events** by **probability redistribution**. Commonly used in bigram and trigram LMs, where many word combination never occurs.

- **Limitations**

- For **small datasets**, the estimates **may not be accurate** as we don't observe events for every c times.



Absolute Discounting

- **Basic idea**

- Just subtract a little from each count.
- Suppose we wanted to subtract a little from a count of 4 to save probability mass for the zeros.
- **How much to subtract?**

- **Church and Gale's clever idea** (1991)

- Divide up 22 million words of AP Newswire to training and held-out set
- For each bigram in the training set, see the actual count in the held-out set!
- It sure looks like the adjusted count is:

$$c^* = (c - 0.75)$$

Bigram count in training	Bigram count in held-out set
0	.0000270
1	0.448
2	1.25
3	2.24
4	3.23
5	4.21
6	5.23
7	6.21
8	7.21
9	8.26

Absolute Discounting Interpolation

(Jelinek and Mercer, 1980s)



- Save ourselves some time and just subtract 0.75 (or some d)!

$$P_{\text{AbsoluteDiscounting}}(w_i \mid w_{i-1}) = \frac{\overset{\text{discounted bigram}}{c(w_{i-1}, w_i) - d}}{c(w_{i-1})} + \overset{\text{Interpolation weight}}{\lambda(w_{i-1})} \overset{\text{unigram}}{P(w)}$$

- (Maybe keeping a couple extra values of d for counts 1 and 2)
- But should we really just use the regular unigram $P(w)$?
 - For trigram, $P(w_3 \mid w_1, w_2)$, $P(w)$ be $P(w_3 \mid w_2)$ (backoff to the lower order)

Kneser-Ney Smoothing I

(R. Kneser and H. Ney, 1995)



- **Better estimate for probabilities of lower-order unigrams!**

- Shannon game: *I can't see without my reading* glasses Kong ?
- “Kong” turns out to be more common than “glasses”
- ... but “Kong” always follows “Hong”
- **The unigram is useful when we haven't seen this bigram!**
 - Instead of using $P(w)$, it may be better to estimate “How likely is w ”?

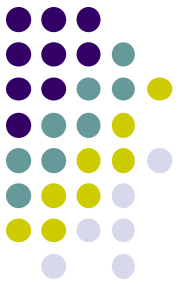
- $P_{\text{continuation}}(w)$: How likely is w to appear as a novel continuation?

- For each word, count the number of bigram types it completes.
- Every bigram type was a novel continuation the first time it was seen

$$P_{\text{continuation}}(w) \propto |\{w_{i-1} : c(w_{i-1}, w) > 0\}|$$

- **Hypothesis**

- Words that have appeared in more contexts in the past are more likely to appear in some new contexts as well.



Kneser-Ney Smoothing II

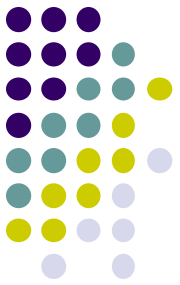
- How many times does **w** appear as a **novel continuation**?

$$P_{CONTINUATION}(w) \propto |\{w_{i-1} : c(w_{i-1}, w) > 0\}|$$

- Normalized by the total number of **word bigram types**:

$$|\{(w_{j-1}, w_j) : c(w_{j-1}, w_j) > 0\}|$$

$$P_{CONTINUATION}(w) = \frac{|\{w_{i-1} : c(w_{i-1}, w) > 0\}|}{|\{(w_{j-1}, w_j) : c(w_{j-1}, w_j) > 0\}|}$$



Kneser-Ney Smoothing III

- The number of # of word types seen to precede w

$$|\{w_{i-1} : c(w_{i-1}, w) > 0\}|$$

- Normalized by the # of words preceding all words:

$$P_{CONTINUATION}(w) = \frac{|\{w_{i-1} : c(w_{i-1}, w) > 0\}|}{\sum_{w'} |\{w'_{i-1} : c(w'_{i-1}, w') > 0\}|}$$

- A frequent word (Kong) occurring in only one context (Hong) will have a low continuation probability.

Kneser-Ney Smoothing IV: Extended Interpolated



$$P_{KN}(w_i | w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - d, 0)}{c(w_{i-1})} + \lambda(w_{i-1})P_{CONTINUATION}(w_i)$$

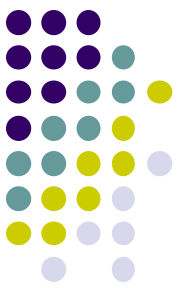
the discount d

λ is the interpolation weight for the lower-order model; a normalizing constant; the probability mass we've discounted

$$\lambda(w_{i-1}) = \frac{d}{c(w_{i-1})} |\{w : c(w_{i-1}, w) > 0\}|$$

the normalized discount d

The number of word types that can follow w_{i-1}
= # of word types we discounted
= # of times we applied normalized discount



Kneser-Ney Smoothing: Recursive Formulation

Recursively calculating the probability of **n-gram** by backing off to lower-order using the same smoothing formula

$$P_{KN}(w_i | w_{i-n+1}^{i-1}) = \frac{\max(c_{KN}(w_{i-n+1}^i) - d, 0)}{c_{KN}(w_{i-n+1}^{i-1})} + \lambda(w_{i-n+1}^{i-1}) P_{KN}(w_i | w_{i-n+2}^{i-1})$$

Lower-order

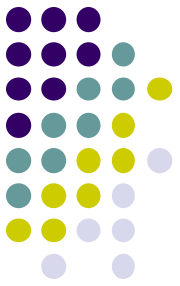
$$c_{KN}(\bullet) = \begin{cases} \text{count}(\bullet) & \text{for the highest order} \\ \text{continuationcount}(\bullet) & \text{for lower order} \end{cases}$$

w_{i-n+1}^{i-1} represents the sequence of words from position $i - n + 1$ to $i - 1$, e.g., for a trigram model ($n=3$) and a sentence “I love dog”, the history w_{i-2}^{i-1} = “I love”, just before w_i (“dog”).

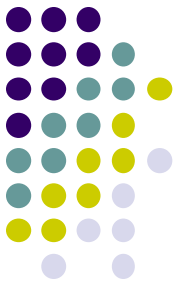
Continuation count = Number of unique single word contexts for •

The commonly used smoothing methods:

- Absolute discounting
- Back-off to lower-order models
- Lower-order context usage (continuation counts)
- Interpolation
- Handling rare or unseen N-grams (recursively backing off, continuation counts)



Practical Issues in N-gram Language Modeling

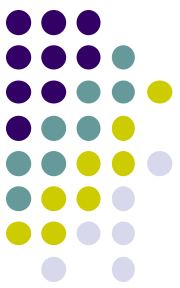


Storage Problems with N-gram LMs

Storage: Need to store count for all n -grams you saw in the corpus.

$$P(w|\text{students opened their}) = \frac{\text{count}(\text{students opened their } w)}{\text{count}(\text{students opened their})}$$

Increasing n or increasing corpus
increases model size!



Huge Scale N-grams

- **How to deal with a large scale of N-grams**

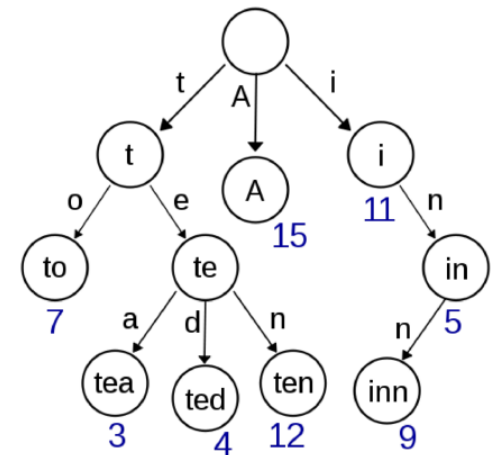
- e.g., Google N-gram corpus

- **Pruning**

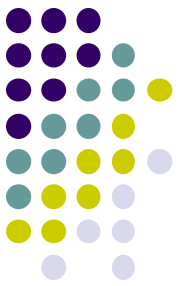
- Only store N-grams with **count > threshold**.
 - Remove singletons of higher-order n-grams
- Entropy-based pruning
 - Remove the N-grams that contributes the least to the overall predictability of the model (by entropy measure)

- **Efficiency**

- **Efficient data structures**, e.g., **trie**
 - Prefix tree, used to store a dynamic set of strings.
 - Each node represents a character of a string
- **Bloom filters**: approximate language models
 - Probabilistic data structure
- **Indexes**, not strings to store words
 - Use Huffman coding to fit large numbers of words into two bytes
- **Quantize probabilities**
 - Use **4-8 bits** instead of 8-byte to store floating point probabilities



<https://en.wikipedia.org/wiki/Trie>

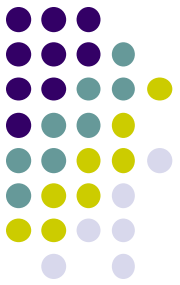


Engineering N-gram Models

- For 5+-gram models, need to store 100M–10B context word-count triples.
- Make it fit into memory by **delta encoding scheme**:
 - Store deltas instead of values and use variable-length encoding.
 - Significantly reduced the memory footprint of N-gram models

Word Context Count								
(a) Context-Encoding			(b) Context Deltas			(c) Bits Required		
<i>w</i>	<i>c</i>	<i>val</i>	Δw	Δc	<i>val</i>	$ \Delta w $	$ \Delta c $	$ val $
1933	15176585	3	1933	15176585	3	24	40	3
1933	15176587	2	+0	+2	1	2	3	3
1933	15176593	1	+0	+5	1	2	3	3
1933	15176613	8	+0	+40	8	2	9	6
1933	15179801	1	+0	+188	1	2	12	3
1935	15176585	298	+2	15176585	298	4	36	15
1935	15176589	1	+0	+4	1	2	6	3

Pauls and Klein (2011), Heafield (2011)

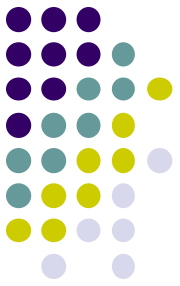


Backoff: for Web-scale N-grams

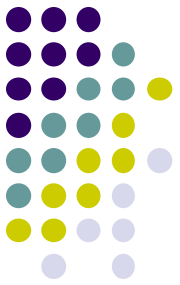
- **“Stupid backoff”** (Brants *et al.*, Google 2007)
 - **Idea:** No discounting, just use relative frequencies
 - Check if the N-gram exists in the training data
 - If yes, use its MLE probability,
 - If not, backoff to a lower-order N-gram, multiply a fixed discount factor $\alpha < 1$ (usually 0.4)
 - Repeat backing off until you reach unigram probabilities

$$S(w_i | w_{i-k+1}^{i-1}) = \begin{cases} \frac{\text{count}(w_{i-k+1}^i)}{\text{count}(w_{i-k+1}^{i-1})} & \text{if } \text{count}(w_{i-k+1}^i) > 0 \\ 0.4S(w_i | w_{i-k+2}^{i-1}) & \text{otherwise} \end{cases} \quad S(w_i) = \frac{\text{count}(w_i)}{N}$$

- Computationally cheap but surprisingly effective at large scale (Google 2007)
- Why it's called stupid?
 - Does **not** produce **probability**, but **scores** and just multiplying a constant factor

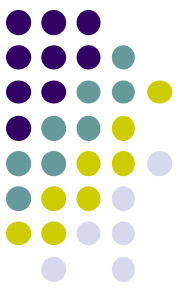


Evaluation of Language Models



Evaluation of N-gram Models

- **Extrinsic evaluation** compares models A and B
 - By putting each model in NLP tasks and getting the results and determine how useful each model is:
 - How many misspelled words corrected properly
 - How many words translated correctly
 - Then, comparing accuracy for A and B
 - **Difficulty of extrinsic evaluation**
 - Time-consuming. may take days or weeks
 - So, **generally only useful in pilot experiments**
 - Bad approximation unless the test data looks **just** like the training data
 - But is helpful to think about.
- **Intrinsic evaluation** measure how good the LM is.
 - How well the LM models a natural language.
 - **LMs** should assign **high probability** to **real language** they have not seen before (test data or new data by generalization).
 - Do similar words have similar probabilities?



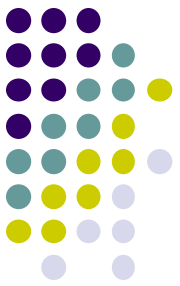
Perplexity: Performance Metric for LM

- **Calculating perplexity** (perplexity “**surprise**”):

- $PP(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}}$ The smaller the better
- $PP(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} = \sqrt[N]{\prod_i^N \frac{1}{P(w_i | w_1 \dots w_{i-1})}}$ by chain rule for joint probability
- $PP(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} = \sqrt[N]{\prod_i^N \frac{1}{P(w_i | w_{i-1})}}$ normalized by the N number of words

- **How well** can the **LM model predict** the **next word**? (the Shannon Game)

- “I always order pizza with cheese and ____”
 - If a model assigns **high probabilities**, then **low perplexity**, **accurate prediction**
low probabilities, then **high perplexity**, **poor prediction**
- **Minimizing the model's** perplexity means **maximizing** the probability for a sequence of words, increasing the prediction capability.



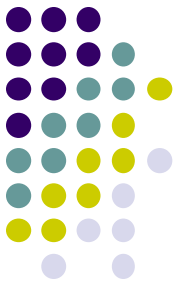
Evaluation of Language Models

N-gram order	Unigram	Bigram	Trigram
Perplexity	962	170	109

Training 38 million words, testing 1.5 million words with WSJ

Model	Num. Params [billions]	Training Time		Perplexity
		[hours]	[CPUs]	
Interpolated KN 5-gram, 1.1B n-grams (KN)	1.76	3	100	67.6
Katz 5-gram, 1.1B n-grams	1.74	2	100	79.9
Stupid Backoff 5-gram (SBO)	1.13	0.4	200	87.9
Interpolated KN 5-gram, 15M n-grams	0.03	3	100	243.2
Katz 5-gram, 15M n-grams	0.03	2	100	127.5
Binary MaxEnt 5-gram (n-gram features)	1.13	1	5000	115.4
Binary MaxEnt 5-gram (n-gram + skip-1 features)	1.8	1.25	5000	107.1
Hierarchical Softmax MaxEnt 4-gram (HME)	6	3	1	101.3
Recurrent NN-256 + MaxEnt 9-gram	20	60	24	58.3
Recurrent NN-512 + MaxEnt 9-gram	20	120	24	54.5
Recurrent NN-1024 + MaxEnt 9-gram	20	240	24	51.3

Table 1: Results on the 1B Word Benchmark test set with various types of language models.



Perplexity Calculation and Interpretations

- **Examples**

- **Q1:** How hard is the task of recognizing digits “0,1,2,3,4,5,6,7,8,9”
 - Suppose a sentence consisting of random digits, where the probability of each digit is $P=1/10$. What is the perplexity of this sentence?

- $PP(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} = (1/10^N)^{-\frac{1}{N}} = 10$

- **Q3:** A traffic signal has three colors: green, yellow, and red, which appear with the following probabilities. What is the perplexity of the sequence (green, yellow, red)?

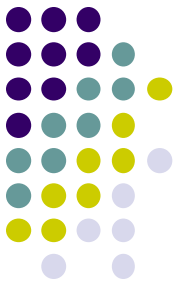
- $P(\text{green}) = 2/5, P(\text{yellow}) = 1/5, P(\text{red}) = 2/5$

- $PP(\text{green, yellow, red}) = \left(\frac{2}{5} \times \frac{1}{5} \times \frac{2}{5}\right)^{-\frac{1}{3}}$

- **Lower perplexity – better model**

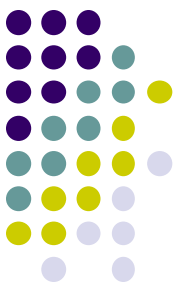
- **Various interpretations of Perplexity**

- Geometric mean, cross-entropy, branching factor



What is *Geometric* Mean?

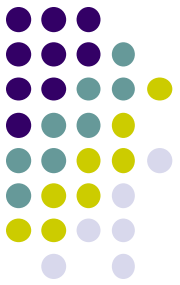
- **Multiplying many numbers** $[0,1]$ **affects** the accuracy measure
 - $P(w_1, w_2, \dots, w_n) \approx \prod_i^n P(w_i | w_{i-k}, \dots, w_{i-1})$
 - Multiplying probabilities of longer sentence will become smaller than shorter sentence.
 - Difficult to compare the predictive accuracy of different length of sentences.
- **Arithmetic mean vs geometric mean**
 - Arithmetic mean: $\frac{(1+1+3+50)}{4} = 13.75$
 - Geometric mean: $(1 \times 3 \times 50)^{1/4} = 3.50$ from $(\prod_i^n x_i)^{1/n} = \sqrt[n]{x_1 x_2 \dots x_n}$
- **When geometric mean is better?**
 - For **multiplicative relationships** than additive, especially numbers $[0,1]$
 - e.g., calculating compound interests
 - It can **avoid bias** toward **extreme values** (**problematic to arithmetic mean**)



Perplexity as a *Cross-Entropy*

- **Given a random variable X** (or a training dataset)
 - **Entropy of X :** $H(X) = \sum_x -P(x) \log_2 P(x)$
 - The entropy of English:

Given the start of a text, can we guess the next word? For **humans**, the measured entropy is only about **1.3**. **What about N-gram models?** LM estimates the probability of word sequences.
 - **Cross-entropy:** $H(P, \hat{P}) = \sum_x -P(x) \log_2 \hat{P}(x)$
 - Measures how close the estimated $\hat{P}$ is to true P (cross-entropy $\geq$ entropy)
- **Deriving Perplexity in terms of cross-entropy**
 - **Perplexity** $= P(\text{sequence})^{-1/n} = (P(w_1)P(w_2) \dots P(w_n))^{-1/n}$
 - Taking **log**: **logPerplexity** $= -\frac{1}{n} \sum_i \log P(w_i)$
 - Then, **Perplexity** $= e^{-\frac{1}{n} \sum_i \log P(w_i)} \simeq 2^{\text{cross-entropy}}$
 - The **cross-entropy** $H(P, \hat{P})$ for a sequence is defined as the **average negative log probability of the correct prediction** $-\frac{1}{n} \sum_i \log \hat{P}(w_i)$ ($P(x) = 1$).



Perplexity as a Branching Factor

- *Perplexity* = $e^{-\frac{1}{n} \sum_i \log P(w_i)} \equiv 2^{\text{cross-entropy}}$
 - Assign probability of 1 to the test data $\rightarrow$ perplexity = 1 (minimum or best)
 - Assign probability of $1/|V|$ to every word $\rightarrow$ perplexity = $|V|$ (random guess)
 - Assign probability of zero to anything $\rightarrow$ perplexity = ∞ (worst)
- **Branching factor**
 - The number of possible options at each decision point (node)
 - So, 6 bits cross-entropy means our model perplexity is $2^6 = 64$
 - Equivalent uncertainty to a uniform distribution over 64 outcomes
 - This is the average branching factor at each decision point.